Lógica, análisis y diseño de la solución Ude@Book

Giselle Juliana Gómez, Joshua Castro Solarte

1 Estructuras de datos y manejo de usuarios

La solución se organizará mediante clases que representen usuarios, amistades, solicitudes, mensajes, álbumes y fotografías. La clase `RedSocial` coordinará las operaciones principales y administrará las estructuras generales del sistema. Cada objeto tendrá un único propietario, mientras que las demás estructuras podrán acceder a él mediante identificadores o referencias, evitando almacenar copias completas de la misma información.

Como el proyecto no contempla registrar ni eliminar usuarios durante la ejecución, la cantidad de usuarios se conocerá al cargar los archivos. Esto permite reservar las estructuras necesarias de acuerdo con el tamaño real de los datos, evitando asignaciones innecesarias y redimensionamientos frecuentes.

2 Matriz de adyacencia dinámica

Elegimos una matriz de adyacencia completa empaquetada en bits porque las consultas de amistad y el cálculo de amigos en común son operaciones centrales del proyecto. Esta estructura permite consultar una relación en tiempo constante y comparar las filas mediante operaciones sobre bloques de bits, facilitando las sugerencias y la selección ponderada de amigos. Aunque requiere aproximadamente el doble de memoria que la representación triangular, su consumo para 1.000 usuarios sigue siendo reducido y permite una implementación más directa. La memoria se reserva dinámicamente según la cantidad de usuarios cargados, y la clase responsable mantiene la simetría mediante actualizaciones de ambas posiciones de cada amistad.

3 Propiedad de los datos y funcionalidades

Para evitar duplicación, los mensajes se almacenarán una sola vez en la colección central administrada por `RedSocial`. Las bandejas de enviados y recibidos podrán mantener identificadores o referencias al mismo mensaje, de modo que cambios como marcarlo como leído se reflejen en el objeto original. De igual manera, las amistades se consultarán en la matriz, sin mantener otra colección equivalente salvo que se justifique por rendimiento.

Cada usuario poseerá sus álbumes y cada álbum sus fotografías. La privacidad del álbum determinará quién puede consultarlo, y la navegación se realizará mediante un carrusel circular que permitirá avanzar, retroceder y modificar los "me gusta". Las fotografías conservarán su información en una sola instancia.

Las sugerencias se obtendrán a partir de amigos de amigos, excluyendo al usuario actual y a sus amigos, y ordenando los candidatos por cantidad de amistades en común. En el inicio de sesión se seleccionarán tres fotografías destacadas según los pesos definidos en el enunciado. La mensajería permitirá enviar textos de hasta 512 caracteres a usuarios que sean amigos, leer mensajes pendientes y buscar mensajes por intervalo de fechas.

4 Persistencia durante la ejecución

La clase `Persistencia` se encargará de cargar la información inicial y conservar los cambios realizados. En lugar de guardar únicamente al cerrar el programa o reescribir todos los datos después de cada operación, se propone combinar un archivo base con un registro incremental de modificaciones.

Las operaciones se validarán y prepararán antes de registrarse. Una vez comprobada la escritura del cambio, se actualizarán las estructuras en memoria y se confirmará la operación al usuario. Las modificaciones que afecten varios elementos, como aceptar una solicitud y crear una amistad, deberán tratarse como una única operación lógica para evitar estados inconsistentes durante la recuperación.

Al iniciar el programa se cargará el archivo base y se reproducirán los cambios posteriores. Periódicamente se generará una nueva instantánea para limitar el crecimiento del registro, que solo se retirará cuando la instantánea esté completa. La estrategia deberá contemplar errores de escritura y registros incompletos.

5 Medición de recursos y organización modular

La eficiencia se evaluará mediante el número de iteraciones y la memoria utilizada por cada funcionalidad, incluidas las operaciones auxiliares y las reservas dinámicas. Esto permitirá identificar los procesos más costosos, especialmente las sugerencias de amistad, las consultas de relaciones y las búsquedas de mensajes.

El sistema se dividirá en módulos con responsabilidades específicas: `Usuario`, `MatrizAmistades`, `SolicitudAmistad`, `Album`, `Foto`, `Mensaje`, `FechaHora`, `Persistencia`, `MedidorRecursos` y `MenuConsola`, coordinados por `RedSocial`. Cada clase tendrá sus archivos `.h` y `.cpp`; el archivo `main.cpp` contendrá únicamente la función principal.

6 Diagrama de clases

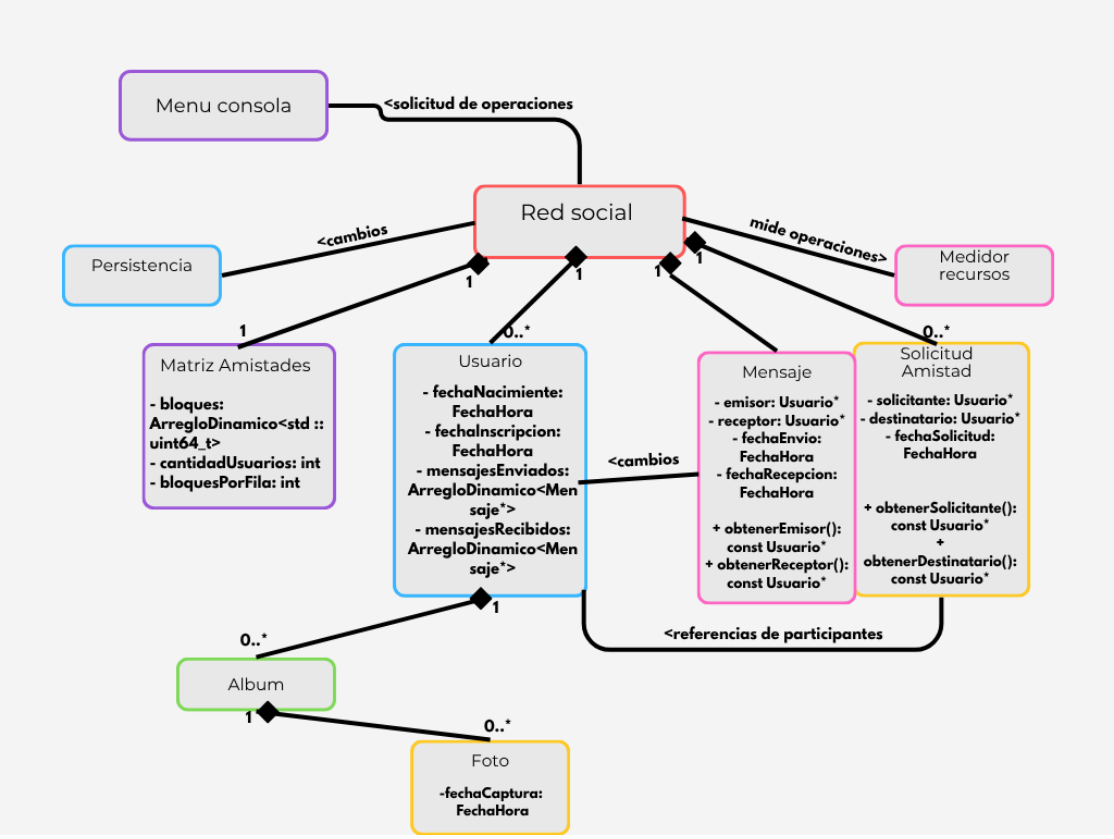


Figure 1: Diagrama de clases